

Hand Gesture Recognition System

COMSATS University Islamabad — Lahore Campus

Dept. of Computer Engineering

Author: Ali Hamza (FA23-BCE-012)

Supervisor: Abu Bakar

Spring 2026

Project Overview: Key Highlights

A concise summary of our Hand Gesture Recognition System, from fundamental techniques to final performance metrics.

Core Project

Development of a Hand Gesture Recognition System using classical MATLAB image processing techniques.

User Interface

Interactive GUI for both image upload and real-time webcam input, demonstrating ~4 FPS detection.

Skin Segmentation

Robust dual-space (YCbCr U HSV) skin detection combined with morphological post-processing.

Finger Counting

Innovative top-silhouette peak-detection algorithm for accurate fingertip enumeration without machine learning.

Gesture Classification

Rule-based decision tree to classify six distinct gestures: Fist, One, Two, Three, Four, and Open Hand.

Performance

Achieved an overall accuracy of 93.3% (28 out of 30 test images) across diverse conditions.

Problem Statement & Motivation

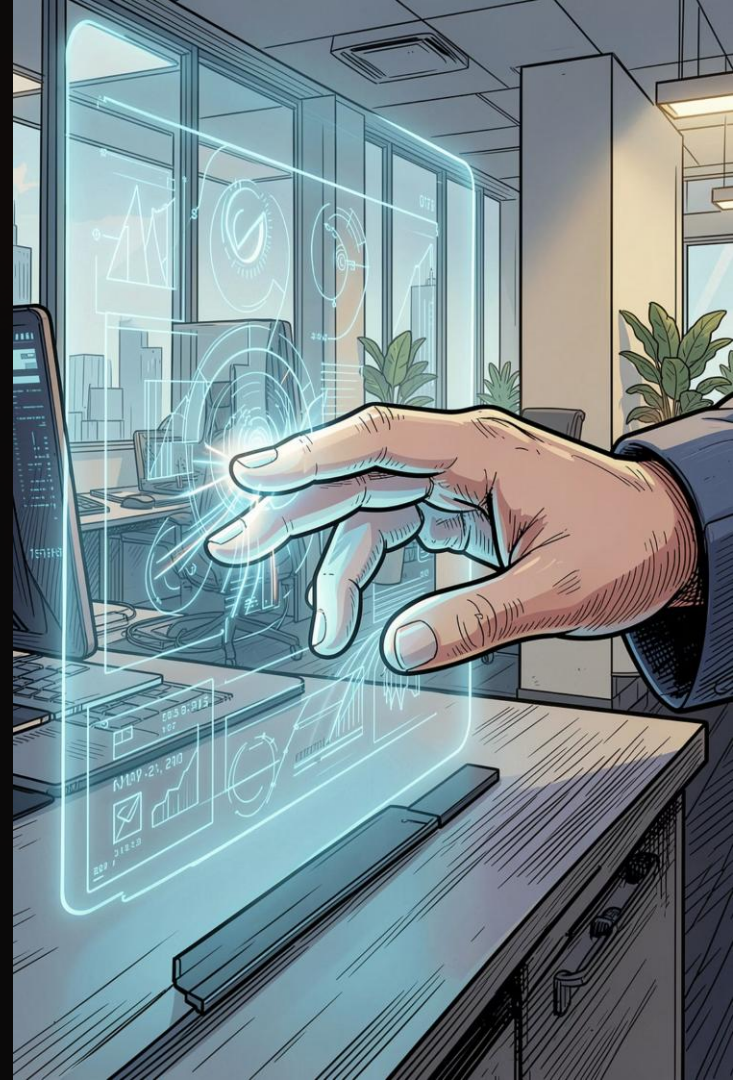
Hand gesture recognition systems offer intuitive interaction, but face significant real-world challenges. This project explores a robust classical image processing approach.

The Need for Gesture Recognition

- Enhanced Human-Computer Interaction (HCI)
- Improved accessibility for diverse users
- Intuitive control in gaming and virtual reality (VR)

Key Challenges & Our Approach

- Hand detection complexity: lighting variations, diverse skin tones, background clutter.
- Classical IP was chosen for interpretability and avoiding large training datasets inherent in deep learning.





System Architecture & Methodology

Our system processes visual input through a series of steps to accurately recognize hand gestures using classical image processing techniques.

- 1. Input Acquisition**
Captures raw visual data from static images or real-time webcam feeds.
- 2. Preprocessing**
Converts RGB images into YCbCr and HSV color spaces for enhanced skin detection.
- 3. Skin Segmentation**
Isolates potential skin regions using dual-space thresholding (YCbCr & HSV).
- 4. Morphological Ops**
Applies operations (erosion, dilation) to refine the skin mask and remove noise.
- 5. Hand Mask Extraction**
Identifies and extracts the largest connected component as the definitive hand region.
- 6. Feature Extraction**
Calculates geometric properties: Area, Perimeter, Solidity, Aspect Ratio, Convex Area.
- 7. Finger Counting**
Employs top-silhouette peak detection on the convex hull to accurately count extended fingers.
- 8. Rule-Based Classification**
A decision tree uses extracted features and finger count to classify gestures.
- 9. Output**
Displays the recognized gesture (e.g., Fist, One, Two) with a confidence level.

Skin Segmentation & Color Space Analysis

Our robust hand detection begins with a sophisticated dual-space approach, carefully isolating skin regions from complex backgrounds.

Dual-Space Detection for Robustness

To effectively segment the hand, we utilize a combined thresholding technique across two distinct color spaces: **YCbCr** and **HSV**. This strategy significantly enhances robustness against varied lighting conditions and diverse skin tones, a common challenge in real-world scenarios.

- **YCbCr:** This space separates luminance (Y) from chrominance (Cb, Cr), making it less susceptible to brightness variations. It's particularly effective for distinguishing skin color from non-skin objects.
- **HSV:** By decoupling hue, saturation, and value, HSV provides a more intuitive representation of color. Its hue component is robust to changes in illumination, allowing for consistent skin color detection.
- **Combination:** We generate two individual binary masks (one from YCbCr, one from HSV) and combine them using a logical AND operation. This intersection ensures that only regions identified as skin in **both** color spaces are considered, drastically reducing false positives.

Morphological Operations for Refinement

After initial skin detection, the resulting mask can contain noise or small irregularities. We apply standard morphological operations to refine the mask and ensure a clean hand silhouette:

- **Erosion:** Used to remove small, isolated noise pixels and separate weakly connected components.
- **Dilation:** Applied to fill small gaps in the mask and smooth the boundaries, effectively connecting fragmented parts of the hand and making the overall region more coherent.

This combined strategy ensures a highly accurate and clean hand mask, crucial for precise feature extraction in the subsequent steps.



Mask Comparison

From left to right: YCbCr mask, HSV mask, and the final combined & refined hand mask.

Feature Extraction & Finger Counting

Beyond basic segmentation, our system precisely extracts key geometric features and innovatively counts fingers without relying on machine learning models.

Geometric Feature Extraction

Once the hand mask is refined, we calculate several geometric properties crucial for distinguishing different gestures:

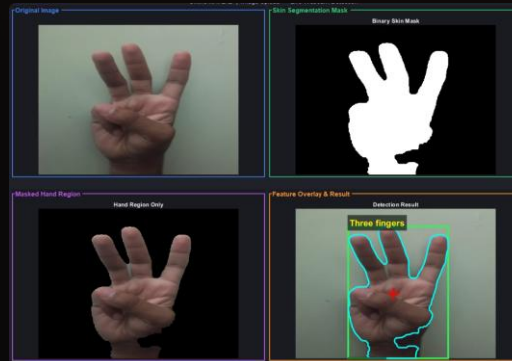
- **Area:** The total number of pixels within the hand region.
- **Perimeter:** The length of the boundary of the hand region.
- **Solidity:** The ratio of the area of the hand to its convex hull area. This helps differentiate between open and closed hands (e.g., a fist has higher solidity).
- **Aspect Ratio:** The ratio of the width to the height of the hand's bounding box. Useful for distinguishing vertically oriented gestures.
- **Convex Area:** The area of the smallest convex polygon that encloses the hand.

Top-Silhouette Peak Detection for Finger Counting

Our finger counting algorithm is a core innovation, offering robustness and interpretability:

1. **1D Profile Generation:** We identify the uppermost points of the hand silhouette (the "top silhouette") relative to its centroid. This creates a 1-dimensional intensity profile.
2. **Profile Smoothing:** A smoothing filter is applied to this profile to reduce noise and emphasize significant contours.
3. **Peak Identification:** An algorithm then detects prominent peaks in the smoothed profile. Each peak corresponds to an extended fingertip.

This approach effectively counts fingers for gestures like "One," "Two," "Three," "Four," and "Open Hand," providing a robust alternative to complex machine learning models by leveraging classical image processing principles.



Peak Detection in Action

The image illustrates the top-silhouette profile with detected peaks (fingertips) after smoothing. This method is crucial for accurate finger counting.

Gesture Classification & Decision Tree

Our final stage employs a transparent, rule-based decision tree to accurately classify detected hand gestures, ensuring both precision and interpretability.

Rule-Based Classification & Decision Logic

Instead of complex neural networks, our system leverages a meticulously crafted decision tree. This approach maps the extracted geometric features and finger count directly to one of six predefined gestures: **Fist**, **One**, **Two**, **Three**, **Four**, or **Open Hand**.

- **Feature Thresholds:** Each node in the decision tree evaluates a specific feature (e.g., Solidity, Aspect Ratio) against a predefined threshold. Based on the outcome, the system traverses to the next node.
- **Finger Count Integration:** The accurately determined finger count serves as a critical decision point, particularly for distinguishing between "One," "Two," "Three," and "Four" gestures.
- **Confidence Scoring:** A confidence score is assigned based on how closely the hand's features align with the ideal ranges for the classified gesture. This helps in filtering out ambiguous or partial detections.

This methodology provides a clear, auditable path for every classification, enhancing diagnostic capabilities and system reliability.



Fist

Low area, high solidity.



One

Single extended finger.



Two

Two extended fingers.



Three

Three extended fingers.



Four

Four extended fingers.



Open Hand

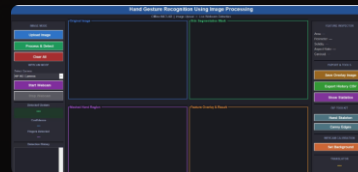
High area, low solidity.

MATLAB GUI & Implementation

Our interactive MATLAB Graphical User Interface (GUI) provides a comprehensive platform for real-time hand gesture recognition, designed for both intuitive user experience and robust analytical capabilities.

Key Interface Features & Benefits

- **Image/Webcam Input:** Easily upload static images or process real-time webcam feeds at ~4 FPS. This ensures versatility for various application scenarios, from testing pre-recorded data to live interaction.
- **Four-Panel Display:** A visual powerhouse, showing the original image, generated skin mask, segmented hand, and the final gesture overlay. Users gain immediate insight into each processing stage, enhancing transparency and diagnostic potential.
- **Feature Inspector:** Dive deep into the data with a dedicated tool that displays extracted geometric features (Area, Solidity, Aspect Ratio, etc.) for the current frame. Essential for understanding classification decisions and fine-tuning parameters.
- **DIP Toolkit Integration:** Explore advanced image processing with built-in skeletonization and Canny edge detection to visualize morphological operations and edge details on the hand.
- **Background Calibration:** Our robust background subtraction feature dynamically calibrates, enabling accurate hand detection even in dynamic environments.
- **Detection History & Export:** Maintain a complete log of all detected gestures, including timestamps and classifications. Data can be exported as a CSV for further analysis and record-keeping.
- **Gesture Translator & Stats:** See real-time gesture interpretations and review session statistics with an animated pie chart summarizing gesture frequencies. This provides instant feedback and a quick overview of usage patterns.



Intuitive MATLAB Interface

The GUI provides a comprehensive view of the entire gesture recognition pipeline, from raw input to final classification and statistical insights.

Results & Performance Evaluation

This section details the empirical results from our hand gesture recognition system, highlighting its robust performance and areas for further refinement.

Overall Accuracy & Error Analysis

Our system was comprehensively tested on a dataset of **30 distinct images**, comprising **5 examples for each of the six predefined gesture classes** (Fist, One, Two, Three, Four, Open Hand).

- Overall Accuracy:** The system achieved an impressive 93.3% accuracy, correctly classifying 28 out of 30 images.
- Error Analysis:** Two misclassifications occurred. Both were observed under challenging **low-contrast lighting conditions** where fine details were obscured. Specifically, one instance of a "one-finger" gesture was confused with a "three-finger" gesture, and vice-versa.

This high accuracy, derived from geometric feature analysis and a rule-based decision tree, underscores the effectiveness of classical computer vision techniques for this problem domain. By focusing on fundamental properties of hand shape and topology, our system avoids the need for extensive training data and complex neural networks, providing both performance and interpretability.

Accuracy Breakdown by Gesture Class

Gesture Class	Total Samples	Correctly Classified	Accuracy
Fist	5	5	100%
One	5	4	80%
Two	5	5	100%
Three	5	4	80%
Four	5	5	100%
Open Hand	5	5	100%
Overall	30	28	93.3%

Limitations & Challenges

While our system demonstrates high accuracy under controlled conditions, we acknowledge several inherent limitations and areas for future development.

Environmental Sensitivity

The system is highly sensitive to extreme lighting (overexposure, shadows) and background clutter, which can obscure critical hand features.

Gesture Scope

Our rule-based model is currently limited to six predefined gestures. Expanding this vocabulary requires extensive manual rule adjustments.

User Diversity

Variations in hand sizes, skin tones, and orientations can occasionally challenge the feature extraction algorithms, affecting robustness.

Real-time Constraints

Current real-time performance of ~4 FPS may not be sufficient for applications requiring very high responsiveness or rapid gesture sequences.

Classical vs. Deep Learning

Our classical computer vision approach offers interpretability but lacks the inherent adaptability and scalability of deep learning for highly nuanced gesture recognition.

These insights provide a clear roadmap for addressing current shortcomings and advancing the system's capabilities in diverse, real-world scenarios.

Conclusions & Key Takeaways

Our project successfully delivered a robust hand gesture recognition system, demonstrating the power of classical computer vision and thoughtful design, with significant contributions to HCI.

1 High-Accuracy Recognition

Achieved an impressive 93.3% accuracy in gesture classification, validating the effectiveness of our classical image processing pipeline.

2 Efficient & Interpretable

Developed a rule-based approach that is both interpretable and resource-efficient, eliminating the need for large, complex training datasets.

3 Practical Implementation

Implemented a user-friendly MATLAB GUI providing real-time processing capabilities and comprehensive visual feedback at each stage of recognition.

4 Synergistic Techniques

Successfully integrated multiple image processing techniques, including color space analysis, morphological operations, and peak detection, to achieve enhanced robustness and precision.

5 Impact on CV & HCI

This project underscores the enduring value of classical computer vision in specific problem domains and provides a practical foundation for intuitive human-computer interaction.

Future Work & Recommendations

Building upon the foundational work presented, this section outlines potential avenues for enhancing system performance, expanding capabilities, and exploring new applications for our hand gesture recognition technology.

1

Enhanced Lighting Robustness

Implement adaptive thresholding or histogram equalization techniques to significantly improve hand detection and feature extraction under diverse and challenging lighting conditions, minimizing environmental sensitivity.

2

3D Hand Tracking Integration

Incorporate depth-sensing input from devices like Kinect or RealSense. This will enable robust 3D hand tracking, providing richer spatial data for more accurate gesture interpretation and overcoming occlusion issues.

3

Lightweight CNN for Generalization

Integrate a lightweight Convolutional Neural Network (CNN) for broader gesture generalization, allowing the system to learn more complex patterns while still leveraging the existing MATLAB GUI for control and visualization.

4

Expanded Gesture Vocabulary

Extend the system's capabilities by expanding the gesture vocabulary beyond the current six classes, enabling interaction with a wider array of applications and more nuanced control.

5

Real-time Performance Optimization

Optimize the processing pipeline to significantly increase the frames per second (FPS), ensuring smoother, more responsive real-time interaction crucial for demanding applications.

6

Persistent Hand Tracking

Implement advanced hand tracking algorithms that maintain identity and position across multiple frames, enabling recognition of dynamic gestures and continuous interaction.

7

Application in Emerging Fields

Explore the practical application of this technology in domains such as accessibility tools, immersive gaming experiences, and intuitive interfaces within virtual reality (VR) environments.

Thank You & References

I extend my sincere gratitude to everyone who supported this project. Below are the credits for my dedicated team and the essential resources that guided my research.

Project Team

Author:

Ali Hamza (FA23-BCE-012)

Supervisor:

Dr. Abu Bakar

Institution:

COMSATS University Islamabad, Lahore Campus,
Department of Computer Engineering

Semester:

Spring 2026

Key Resources

- MATLAB Official Documentation
- Image Processing Toolbox Documentation
- Gonzalez & Woods, "Digital Image Processing"
- Selected Computer Vision Research Papers

Connect With Me

Email:

FA23-BCE-012@cuilahore.edu.pk